Anil Seth

Getting a Hang of Zope's Grok

According to Wikipedia, 'to grok' is to share the same reality or line of thinking with another physical or conceptual entity. For example, to say that you 'know' LISP is simply to assert that you can code in it if necessary—but to say you 'grok LISP' is to claim that you have deeply imbibed the world-view and spirit of the language, with the implication that it has transformed your view of programming.

Getting started with Django, or for that matter, Rails, typically involves creating a model. This is likely to be just a relational table. The framework creates a form with the option to insert, edit and delete records. And you have a working application in a matter of minutes.

The Grok tutorial (http://grok.zope.org/doc/current/tutorial.html) comes as a surprise; it starts with views and templates instead. Persistence of data 'magically' happens behind the scenes, using ZODB. The tutorial exposes you to the basic usage of Zope Page Templates and its relationship to the Python code. You will get a flavour of Grok quickly, though building a working application takes more effort.

Comparing it with the MVC paradigm, one can relate to the idea of models and views—but controllers seem to be missing (see http://grok.zope.org/doc/current/grok_overview.html). The URLs point to model objects in the Grok application views. This brings up an interesting issue when trying different frameworks: while it may take just a few lines of code to implement some concept, it can be hard, in the beginning, to find what those lines are, or where they go! So, the chances are that the first framework you learn will feel much easier and more 'natural' than the rest!

While Grok makes it easier to build Zope3 applications, an understanding of the Zope concepts is necessary. Zope is sufficiently different, so you won't be surprised to come across a page like http://wiki.zope.org/zope3/ZopeInAnger, which says, "ZopeInAnger is meant to help you develop Zope programs without necessarily understanding the full API (as if that were possible for mere mortals)..." The geeky humour ensured that I did not give up.

While Grok's tutorial was fine, several user-contributed example applications had issues with current versions. The issues were minor, but can be intimidating when one is just getting started.

Persistence

The main aspect of Grok that is conceptually different from other major frameworks is the persistence mechanism of Grok/Zope3, which can be thought of as a database consisting of containers and persistent objects. A container can be thought of as a Python dictionary with key-value pairs, much like gdbm or Berkeley DB. The value in this case is any Python object that can be pickled. The key is used to navigate through the objects. While Zope applications normally use ZODB, it is possible to replace ZODB with a relational database and an object-relational mapper, like SQLAlchemy.

A persistent object is defined by a class that inherits from *grok.Model*. Consider a simple example like an 'LFY' application that stores articles. A code fragment may look like what's shown below:

```
class LFY(grok.Application, grok.Container):
    def add_object(self, name, article):
        self[name] = article

class Article(grok.Model):
    title = ''
    content = ''
```

If you want to dynamically create a form for the model, you need to make more of an effort. To define the object in a Zope interface, with the specification of the schema data types, let's first implement the interface in the model. The code fragment for the interface should be as follows: